SureInspect: Master Plan for the OpenCV AI Competition 2026

An explainable, risk-controlled agentic quality inspection system built on OpenCV 5 and AWS

Working name: **SureInspect** (rename freely). Final deadline: **October 26, 2026, 11:59 p.m. Pacific Time** (October 27, about 12:30 p.m. IST) (OpenCV Competition). Time available: about 10 days, roughly 70–80 working hours.

Contents

1. What we are building (one paragraph)
2. Research: what already exists
3. Research: what does not exist yet (the gaps)
4. What we combine, and what we add that is new
5. Final system design
6. Path to victory: feature → rubric mapping
7. Feature priority: Must / Should / Could
8. Step-by-step build plan (day by day)
9. Evaluation plan (the experiments judges will see)
10. Submission checklist
11. Risks and fallbacks
12. Honest claims: what we can and cannot say

---

1. What We Are Building

SureInspect inspects product images from MVTec AD. It does not just output a defect heatmap. An agent looks at the first result and **decides what to do next**:

- Pass the part right away.
- Gather more evidence (zoom in, compare with a defect-free reference, check the lighting).
- Fail the part.
- Send it to a human.

Every decision uses a **statistically calibrated threshold**, so we can state and measure the maximum false-alarm rate on good parts. Each human review **immediately improves the detector without retraining**. Every step is shown with OpenCV measurements and a visual trace.

Short pitch for judges:

> "Most inspection AI gives a score and stops. SureInspect decides when it is unsure, gathers more visual evidence with OpenCV 5, keeps false alarms under a calibrated limit, explains every decision with measurements, and learns from each human review without retraining."

---

2. Research: What Already Exists

2.1 Datasets

Dataset	What it has	Use for us
MVTec AD	5,354 images, 15 categories; defect-free training images; 1,888 pixel-accurate defect masks (MVTec AD paper)	**Main dataset**
MVTec AD 2	8,000+ images, 8 harder scenarios, lighting changes in the test sets (MVTec AD 2)	Optional robustness demo (public test set only)
NEU-DET	1,800 steel images, 6 classes, bounding boxes (NEU-DET)	Skip (time)
Real-IAD	Multi-view (5 cameras), gated access, CC BY-NC-SA (Springer, Real-IAD FDR study)	Skip (access and time)

2.2 Detection models (perception)

Model	Key facts	Use for us
PatchCore	Memory bank of normal patch features with nearest-neighbor scoring (Anomalib docs); about 99.6% image AUROC on MVTec AD (Codesota guide)	**Main detector**
EfficientAD	About 99.8% image AUROC, faster (Codesota guide)	Backup
AnomalyDINO	DINOv2-based few-shot detector, available in Anomalib (Anomalib AnomalyDINO)	Optional: “new product with only a few good images”
Anomalib	Trains on MVTec and exports Torch, ONNX or OpenVINO (Anomalib docs)	Training tool

2.3 Agentic and explainable inspection research

Work	What it does	Limitation for us
AgentIAD	VLM agent with Zoom, Compare-to-normal and Terminate tools; 95.3% image AUROC on MVTec AD (AgentIAD)	Needs VLM fine-tuning plus reinforcement learning; uses heuristic final decisions
ActiveInspect	Inspection as **budget-constrained** evidence selection (choose view, zoom, compare-normal, terminate). Keeps 99.8% of exhaustive-scan AUROC using 18% of the observations, 5.3× faster (ActiveInspect)	VLM policy trained with GRPO; no statistical false-alarm guarantee; no human feedback loop
VLA-AD	Perception, Reasoning and Action agents; context-aware thresholds cut false positives by 34.7% (VLA-AD)	Fine-tuned LLM; continual learning retrains adapters

Work	What it does	Limitation for us
AnomalyGPT	LVLM with localization; 94.1% image AUROC one-shot (AnomalyGPT)	Heavy GPU; slow
LogicAD	Logic rules for missing or misplaced parts (LogicAD)	Targets MVTec LOCO
MMAD benchmark	Shows GPT-4o alone averages only 74.9% on industrial anomaly questions (MMAD)	Our evidence that VLMs need real vision tools

2.4 Reliability and human-in-the-loop research

Work	What it does	Use for us
Training-free HITL PatchCore correction	A reviewer marks a false alarm as normal, and that image's novel patches are inserted into PatchCore's memory bank. No retraining or gradients; threshold recomputed after each correction (arXiv 2608.17775)	**Combine**: HUMAN decisions improve the detector
Risk-controlled detection (conformal + per-category calibration)	Split-conformal p-values, per-category ("Mondrian") calibration, finite-sample false-discovery-rate guarantees (Springer, Real-IAD FDR)	**Combine**: calibrated PASS/FAIL thresholds
Conformal segmentation for surface defects	Conformal risk control on pixel masks for FDR/FNR, tested on NEU-DET (arXiv 2504.17721)	Supporting reference

2.5 Robustness evidence

- On MVTec AD 2, methods that score over 90% AU-PRO on MVTec AD reach at most **58.7%**. With unseen lighting, EfficientAD drops from 30.8% to **19.2%** on the strict metric (MVTec AD 2 paper).
- **Meaning:** lighting changes break detectors. An agent that **notices a lighting shift and reacts** fills a real gap.

2.6 OpenCV 5 and AWS building blocks

Capability	Source
New graph DNN engine; over 80% ONNX operator coverage; DINOv2-small 24.4% faster than ONNX Runtime on CPU	OpenCV 5 Deep Dive
Learned matchers (ALIKED, DISK, LightGlue) plus classic ORB/SIFT for alignment	OpenCV 5 Deep Dive
Native LLM/VLM inference (Qwen 2.5, PaliGemma) through the `Net` API	OpenCV 5 Deep Dive
COOL on AWS Graviton: about 20% average speedup, up to 70% on some functions	AWS COOL blog
A generic OpenCV MCP server exists (filters, edges, detection, tracking), with no inspection-specific tools	OpenCV MCP Server

2.7 What competitors are building

The Devpost project gallery **is not published yet** (Devpost gallery), so other entries cannot be checked. The Devpost listing shows **4,364 participants** (Devpost). Expect many simple "upload, then classify" projects. Our edge must come from the agent loop, calibrated guarantees and measured evaluation.

---

3. Research: What Does Not Exist Yet (Gaps)

These are gaps **in the sources reviewed above**. They are not claims that no one, anywhere, has done them.

#	Gap	Evidence
G1	**Agentic inspectors use heuristic stopping and decision thresholds.** None of the reviewed agent systems state a guaranteed false-alarm limit.	AgentIAD, ActiveInspect and VLA-AD report AUROC and F1 but no finite-sample guarantee (AgentIAD, ActiveInspect, VLA-AD)
G2	**Conformal and risk-control methods are single-pass.** They do not decide to gather more evidence when uncertain.	The calibration layer wraps a scorer and selects; there is no active evidence step (Springer FDR)
G3	**Agentic systems need trained VLM policies** (SFT plus RL) and large GPUs, which students cannot reproduce in 10 days or deploy cheaply.	AgentIAD uses SFT plus GRPO; ActiveInspect uses PA-SFT plus GRPO
G4	**Human review is a dead end.** In most systems the HUMAN outcome does not improve the model right away.	VLA-AD retrains adapters in batches; the training-free HITL method is not inside an agent loop (arXiv 2608.17775)
G5	**Lighting shift is a known failure** but no reviewed agent explicitly detects it and changes its plan.	MVTec AD 2 lighting results (MVTec AD 2 paper)
G6	**VLM explanations are not grounded in measurements**, so they can describe defects that are not there.	MMAD shows weak standalone VLM accuracy (MMAD)
G7	**No inspection-specific OpenCV tool server** for agents. The existing OpenCV MCP server is generic.	OpenCV MCP Server

---

4. What We Combine, and What We Add That Is New

4.1 The combination

Ingredient taken from	What we borrow	How we change it
ActiveInspect / AgentIAD	Budgeted evidence loop: zoom, compare-to-normal, terminate	Replace the trained VLM policy with a **transparent policy over OpenCV 5 tools**: no RL, runs on CPU, fully explainable

Ingredient taken from	What we borrow	How we change it
Conformal risk control	Calibrated thresholds with a guaranteed false-alarm rate	Use it to drive a **three-way decision** (PASS / INVESTIGATE / FAIL) inside the agent loop
Training-free HITL PatchCore	Memory-bank insertion on false alarms	Wire it to the **HUMAN outcome**, and recalibrate thresholds after every update
MVTec AD 2 lighting findings	Lighting breaks detectors	**Lighting-shift tool**: detect, then normalize, re-capture or escalate
LogicAD / metrology	Rules and measurements	**Grounded explanation**: the LLM may only describe OpenCV-measured facts
OpenCV MCP idea	Agent-callable vision tools	**Inspection-specific MCP tool server** (stretch goal)

4.2 Our new contributions (what we claim)

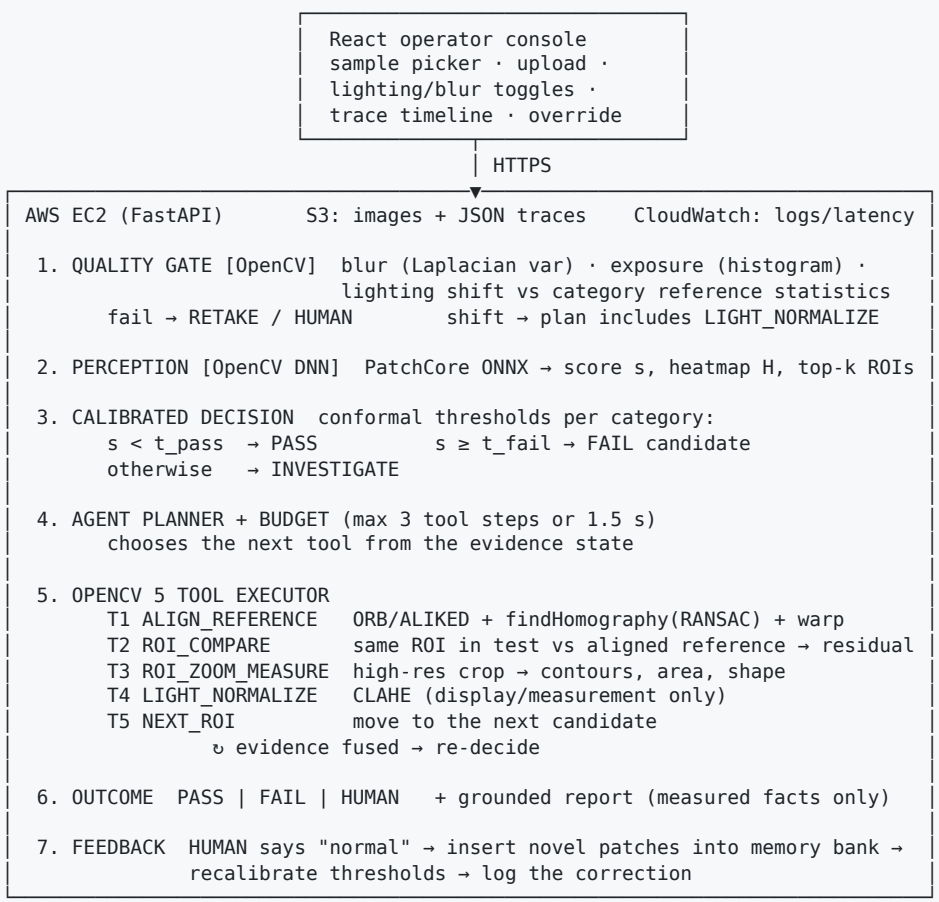
1. **Risk-controlled agentic inspection.** The agent spends extra evidence only when the calibrated decision is uncertain, and its false-alarm rate on good parts is bounded by a target level α that we measure.
2. **Deterministic, explainable policy over OpenCV 5 tools.** It needs no VLM training, runs on CPU, and every step is traceable.
3. **Closed human feedback loop.** HUMAN reviews update the PatchCore memory bank instantly and the thresholds are recalibrated.
4. **Lighting-aware re-planning.** A detected lighting shift changes the plan.
5. **Measurement-grounded reports.** Defect location, size and shape come from OpenCV; the LLM only puts them into words.

4.3 Why this wins over each alternative

Competitor type (likely)	Our advantage
“Upload, then classify” demo	We have a multi-step loop, calibrated guarantees and real evaluation
A chatbot explaining a heatmap	The rules say this does not qualify for the Agentic Vision Award (OpenCV Competition); our evidence changes the next action
A VLM-only inspector	It is slow and ungrounded (MMAD: 74.9% for GPT-4o); ours is grounded in OpenCV measurements
A research-heavy RL agent	It is hard to deploy and reproduce; ours is reproducible on a cheap AWS instance

5. Final System Design

5.1 Architecture



5.2 Calibration (how the guarantee works)

- For each category, hold out about 20% of the **defect-free training images** as a calibration set (for example, about 40 images for `bottle`).
- Score them with PatchCore to get normal scores $s_1, \ldots, s_n$.
- The PASS/FAIL boundary for target false-alarm rate α is the $\lceil (n+1)(1-\alpha) \rceil$-th smallest calibration score. If calibration and test normals are exchangeable, then $P(\text{good part flagged}) \leq \alpha$. This is the standard split-conformal result.
- **Three-way decision:**
  - t_{fail} = threshold at α = 0.01 (very confident it is abnormal)
  - t_{pass} = threshold at α = 0.20 (comfortably normal below it)
  - Scores between them → **INVESTIGATE** (the agent loop runs).
- After every human correction to the memory bank, **recompute the thresholds**. The bank changed, so the old calibration no longer applies.
- **What the guarantee does not cover:** lighting shift breaks exchangeability. That is why the quality gate checks lighting first. Say this openly in the report.

5.3 Agent policy (rules, fully explainable)

Evidence state	Next action
Quality gate failed (blur or exposure)	Stop → **RETAKE / HUMAN**
Lighting shift detected	Mark the guarantee “not valid”; run T4 for display; continue but lean toward **HUMAN**
$s < t_{\text{pass}}$	**PASS**
$s \geq t_{\text{fail}}$ and T2 residual confirms the ROI	**FAIL**
$s \geq t_{\text{fail}}$ but T2 residual is low (disagreement)	T3 zoom/measure → re-decide; still conflicting → **HUMAN**
INVESTIGATE, no alignment yet	T1 ALIGN_REFERENCE → T2 ROI_COMPARE on the top ROI
INVESTIGATE, residual high on the ROI	T3 measure; if area or shape exceed category rules → **FAIL**
INVESTIGATE, residual low on the ROI	T5 NEXT_ROI (if budget remains)
Budget exhausted and still uncertain	**HUMAN**

Every row writes one entry to the JSON trace: `{step, tool, reason, inputs, outputs, ms}`.

5.4 Grounded report

The LLM (Amazon Bedrock, optional) receives **only** the JSON of measurements, for example:

```
{"category":"bottle","decision":"FAIL","roi":[412,88,61,23],
 "area_pct":0.9,"elongation":3.1,"residual_mean":0.42,
 "calibrated_alpha":0.01,"steps":["ALIGN_REFERENCE","ROI_COMPARE","ROI_ZOOM_MEASURE"]}
```

Instruction: “Write two sentences. Use only these numbers. Do not guess the cause.” If Bedrock is not used, a simple text template produces the same report.

5.5 Repository structure

```
sureinspect/
├── README.md                # setup, run, deploy, test
├── requirements.txt         # pinned versions
├── train/
│   ├── train_patchcore.py   # Anomalib training per category
│   └── export_onnx.py
├── core/
│   ├── quality_gate.py      # blur, exposure, lighting shift
│   ├── perception.py        # OpenCV DNN PatchCore inference
│   ├── calibration.py       # conformal thresholds
│   ├── tools.py             # T1–T5 OpenCV tools
│   ├── agent.py             # policy, budget, trace
│   ├── feedback.py          # memory-bank update + recalibration
│   └── report.py            # grounded explanation
├── api/main.py              # FastAPI
├── web/                     # React console
├── eval/
│   ├── run_baseline.py
│   ├── run_agent.py
│   ├── run_corruptions.py
│   ├── run_feedback.py
│   └── make_tables.py
├── deploy/                  # Dockerfile, EC2 setup, CloudWatch config
└── docs/                    # report, diagrams, failure cases
```

6. Path to Victory: Feature → Rubric Mapping

6.1 Overall Awards (100 points) (OpenCV rules)

Criterion	Weight	What earns the points
Technical execution	30	PatchCore running in OpenCV 5 DNN; five OpenCV tools; calibration; clean evaluation with a held-out split
Innovation	20	Risk-controlled agent loop + instant human feedback + lighting-aware re-planning
Real-world impact	20	Fewer false rejects (scrap cost), a measured false-alarm limit, an auditable trace, a feedback loop for operators
User experience	10	Sample picker, one-click corruption toggles, trace timeline, override button
Documentation and presentation	10	Report with all 7 required sections, diagram, pinned deps, 5-minute video
Cloud, reproducibility, responsible operation	10	EC2 + S3 + CloudWatch, Dockerfile, latency logs, human-control path, honest limitations

6.2 Agentic Vision Award (OpenCV rules)

Criterion	Weight	What earns the points
OpenCV 5 + agent integration	30	Every agent decision is triggered by an OpenCV output (score, residual, contour area, blur value)
Orchestration and autonomy	25	Budgeted multi-step loop; acts on its own when confident, escalates when not
Task effectiveness and evaluation	20	Baseline vs agent table; false-alarm rate vs target α; corruption tests
Failure handling, observability, security, human control	15	Quality gate, lighting-shift detection, JSON traces in S3, override plus feedback
UX, docs, demo	10	Trace timeline in the UI and the video

Required evidence for the award (OpenCV Competition): - An agent workflow diagram (Section 5.1) - A trace showing OpenCV output changing a later decision (the JSON trace, shown in the UI and the video) - Evaluation of task success, failure handling, observability and human control (Section 9)

6.3 Best Use of COOL Award (optional)

Only if Days 1–8 finish on time: run the same pipeline on a Graviton instance with COOL, and report a benchmark of preprocessing, alignment and contour time against standard OpenCV (AWS COOL blog). The award requires verified COOL execution on Graviton (OpenCV Competition).

---

7. Feature Priority

Must (the MVP; without these, do not submit)

1. PatchCore for 3 categories (bottle, metal_nut, screw) running in OpenCV 5 DNN
2. Quality gate (blur, exposure)
3. Conformal three-way thresholds
4. Tools T1 ALIGN_REFERENCE, T2 ROI_COMPARE, T3 ROI_ZOOM_MEASURE
5. Agent policy with budget plus JSON trace
6. FastAPI + simple React page
7. AWS EC2 + S3 + CloudWatch
8. Evaluation E1 (baseline vs agent) and E2 (false-alarm guarantee)
9. Report, diagram, video

Should (strongly recommended)

10. Lighting-shift detection + T4 (Evaluation E3)
11. Human feedback: memory-bank update + recalibration (Evaluation E4)
12. Grounded report text (template, or Bedrock)

Could (only if ahead of schedule)

13. MCP server exposing T1–T5 as inspection tools
14. COOL on Graviton benchmark
15. AnomalyDINO “new product with 4 good images” mode
16. MVTec AD 2 public test set demo

---

8. Step-by-Step Build Plan

Assumes about 7–8 hours per day. Dates are in IST.

Day 1: Oct 17, Model and the OpenCV 5 runtime (critical risk day)

1. Set up the environment: Python 3.11, OpenCV 5, Anomalib, pinned `requirements.txt`.
2. Download MVTec AD; keep `bottle`, `metal_nut`, `screw`.
3. Split each category’s `train/good` images 80/20 into **train** and **calib**.
4. Train PatchCore per category on the 80% split.
5. Export to ONNX.
6. Load with `cv.dnn.readNetFromONNX`; confirm OpenCV gives the **same scores** as Anomalib on 20 images.

7. **If loading fails:** export only the WideResNet50 feature extractor to ONNX, run it in OpenCV DNN, and do the nearest-neighbor search over the memory bank in NumPy (also useful later, because Day 6's feedback feature needs direct access to the bank).

- **Done when:** one script prints score + heatmap for any image using OpenCV DNN.

Day 2: Oct 18, Quality gate, calibration, heatmap ROIs

1. `quality_gate.py`: Laplacian variance (blur), histogram percentiles (dark/bright), mean/std of the LAB L channel compared with the category's reference statistics (lighting shift).
2. `calibration.py`: score the calib images; compute t_{pass} (α = 0.20) and t_{fail} (α = 0.01) per category.
3. Heatmap → threshold → `cv.findContours` → top-k ROIs with area and bounding box.

- **Done when:** each image returns `quality, score, zone (PASS/INVESTIGATE/FAIL), rois`.

Day 3: Oct 19, OpenCV tools T1–T5

1. T1: pick the nearest defect-free reference (smallest global feature distance), align with ORB + `cv.findHomography` (RANSAC) + `cv.warpPerspective`; report the inlier ratio. **Low inliers means alignment failed**, which is itself evidence.
2. T2: crop the same ROI from the test and aligned reference images; compute a residual (absolute difference after Gaussian blur, or SSIM).
3. T3: crop the ROI from the full-resolution image; `cv.minAreaRect`, area %, elongation, distance to edge.
4. T4: CLAHE in LAB space, **for display and T3 only, never fed back into PatchCore**.
5. T5: next ROI.

- **Done when:** each tool runs on a test image and returns JSON plus an overlay image.

Day 4: Oct 20, Agent policy, budget, trace

1. `agent.py`: implement the Section 5.3 table as code; budget = 3 tool steps or 1.5 s.
2. Every step appends to the trace with its reason (for example `"score 0.61 in INVESTIGATE zone"`).
3. `report.py`: template text built from the measurements.
4. Run on 30 images; check the traces by eye.

- **Done when:** `python run_agent.py --image X` prints the decision and the full trace.

Day 5: Oct 21, API + web console

1. FastAPI endpoints: `/inspect`, `/samples`, `/feedback`, `/trace/{id}`.
2. React page with three panels:
   - **Input:** sample picker, upload, toggles (add blur, darken, glare)
   - **Trace:** timeline of steps, each with its overlay image (heatmap, aligned reference, ROI crop, contours)
   - **Result:** PASS/FAIL/HUMAN, the report, calibrated α, latency, and "Mark as normal / defect" buttons

- **Done when:** the full flow works on localhost.

Day 6: Oct 22, Human feedback + AWS deployment

1. `feedback.py`: on "mark normal", insert the image's novel patches (beyond the median nearest-neighbor distance) into the bank (arXiv 2608.17775), then recalibrate.
2. Dockerfile; deploy on EC2; store images and traces in S3; send logs and latency to CloudWatch.

3. Restrict uploads (size and type), add basic rate limiting, keep no personal data.

- **Done when:** a public HTTPS URL works for judges.

Day 7: Oct 23, Evaluation runs

Run E1–E4 (Section 9) on the held-out split; save CSV files and charts.

Day 8: Oct 24, Failure cases + polish (+ optional COOL)

1. Choose 4 documented failure cases with images.
2. Fix the top bugs found during evaluation.
3. Optional: COOL benchmark on Graviton.

Day 9: Oct 25, Technical report + diagrams

Report sections (all required) (OpenCV Competition): problem and users, architecture, OpenCV 5 implementation, AWS deployment, evaluation, limitations, responsible use.

Day 10: Oct 26, Video + submit (by evening IST, well before the deadline)

- 5-minute video (outline below), README check, submit on Devpost.

Buffer: the real deadline is Oct 27, 12:30 p.m. IST. Keep that morning as emergency time only.

---

9. Evaluation Plan

Data split (avoid leakage)

- **Train:** 80% of `train/good`. **Calib:** 20% of `train/good`.
- **Test:** the official MVTec AD test set for the 3 categories.
- For the feedback experiment (E4): split the test set 70/30 into a review pool and a held-out set; report only on the held-out set, following the protocol in arXiv 2608.17775.
- Write this split clearly in the report. The rules prohibit misrepresenting results (OpenCV rules).

Experiments

ID	Question	Compare	Metrics
E1	Does the agent loop help?	Single-pass PatchCore at a fixed threshold vs the full agent	Accuracy on auto-decided parts, false-alarm rate on good parts, defect recall, **HUMAN rate**, average steps, latency
E2	Does the calibrated guarantee hold?	Target α vs measured false-alarm rate on good test parts	Measured vs target at α = 0.01, 0.05, 0.10, 0.20
E3	Does lighting/blur handling help?	Agent with vs without the quality gate on corrupted test images (gamma 1.5, blur σ = 1.0, glare)	Wrong auto-decisions, HUMAN/RETAKE rate
E4	Does human feedback help?	Before vs after N = 5, 10, 20 corrections	False-alarm rate and recall on held-out images

ID	Question	Compare	Metrics
E5 (optional)	COOL speedup	Standard OpenCV vs COOL on Graviton	ms per stage

Rules for honest reporting

- **Always report the HUMAN rate alongside accuracy.** A system that sends everything to HUMAN looks error-free.
- Report the number of images in every table.
- Show failure cases, for example:
  1. Alignment fails on a rotated metal_nut (homography limits).
  2. Tiny screw-thread defects missed at 256 px.
  3. A lighting shift that the gate did not catch.
  4. A false alarm on dust.

---

10. Submission Checklist

From the official requirements (OpenCV Competition):

- ☐ Technical report: problem and users, architecture, OpenCV 5 implementation, AWS deployment, evaluation, limitations, responsible use
- ☐ Code repository accessible to judges
- ☐ Pinned dependencies + build, deploy and test instructions
- ☐ Architecture diagram showing the OpenCV 5, AWS and agent components
- ☐ Working web endpoint (or arranged live demo)
- ☐ Video of 5 minutes or less: team, app working, architecture, main results
- ☐ Evaluation evidence including failure cases
- ☐ **Agentic Vision extras:** workflow diagram, a trace showing OpenCV output changing a decision, evaluation of success, failure handling, observability and human control
- ☐ (Optional COOL extras: version, instance config, benchmark method, proof COOL ran the workload)

Video outline (5:00)

Time	Content
0:00–0:30	Team + problem: false rejects and missed defects cost money; AI that “just gives a score” is not trusted
0:30–1:00	Architecture diagram
1:00–2:45	Live demo: (1) a clear PASS, (2) an uncertain case where the agent aligns, compares, measures and FAILs, (3) darkening the image so the gate catches it and sends it to HUMAN, (4) operator marks a false alarm as normal and the re-score drops
2:45–4:00	Results: E1 table, E2 chart (measured vs target α), E3 and E4

Time	Content
4:00–4:40	Failure cases + limitations
4:40–5:00	AWS deployment + closing line

11. Risks and Fallbacks

Risk	Signal	Fallback
PatchCore ONNX will not load in OpenCV 5	Day 1 error	Feature extractor in OpenCV DNN + NumPy nearest-neighbor search; or `ENGINE_CLASSIC` / `ENGINE_ORT` (OpenCV 5)
The agent does not beat the baseline	E1 shows no gain	Report it honestly; emphasize the guarantee (E2), robustness (E3) and feedback (E4)
Alignment fails on rotated objects	Low inlier ratio	Treat it as evidence (“alignment failed” → HUMAN); limit T1 to bottle/screw
Behind schedule on Day 6	Feedback or AWS not done	Drop MCP/COOL/AnomalyDINO; ship the Must list
AWS cost	—	One small EC2 instance; stop it after judging; use free-tier credits (OpenCV Competition)
Too slow	> 2 s per image	Lower resolution, cache reference features, smaller coreset

12. Honest Claims

We can say: - “False-alarm rate on good parts is calibrated to α under the exchangeability assumption; measured results are in E2.” - “The agent requests more evidence only in the uncertain zone; average steps were X.” - “Human corrections update the detector without retraining; held-out false alarms changed by X.”

We must not say: - “Guaranteed defect-free” or “100% accurate.” - “First ever” or “no one has done this.” Say instead: “In the work we reviewed, we did not find a system combining these.” - That the guarantee holds under lighting shift. It does not, and that is why the gate exists. - Any number we did not measure ourselves.

Sources used in this plan

All sources are linked inline above. Key references: OpenCV AI Competition 2026, Official rules, AgentIAD, ActiveInspect, VLA-AD, Training-free HITL PatchCore, Risk-controlled detection on Real-IAD, MVTec AD 2 paper, OpenCV 5 Deep Dive, AWS COOL blog.